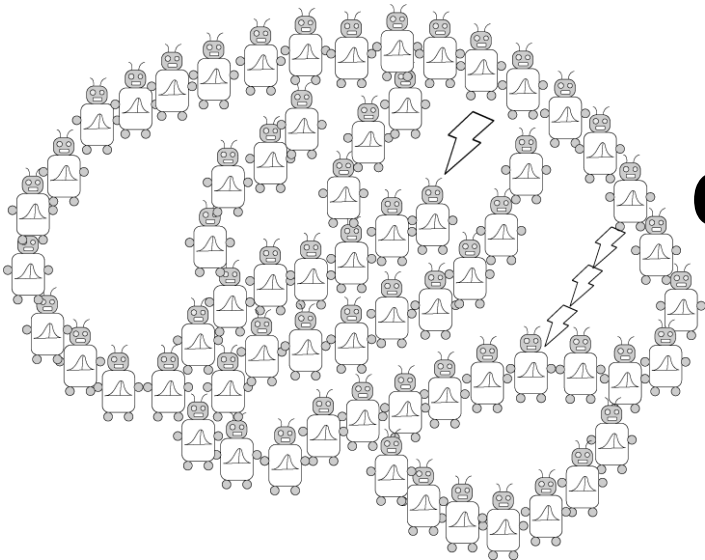


Statistical Decision Theory, Markov Decision Processes, and Reinforcement Learning



**PSYCH/COMP SCI/BMI 841:
Computational Cognitive Science**

Joseph Austerweil

March 20, 2025

- Admin Matters
- Statistical Decision Theory
- Markov Decision Processes
- **Short rest 1**
- Three student presentations:
 1. Schulz et al 1997 by Ashwini
 2. Ho et al 2015 by Khailanii

Short rest 2

3. Ho et al 2017 by Anirudh

Administrative Matters

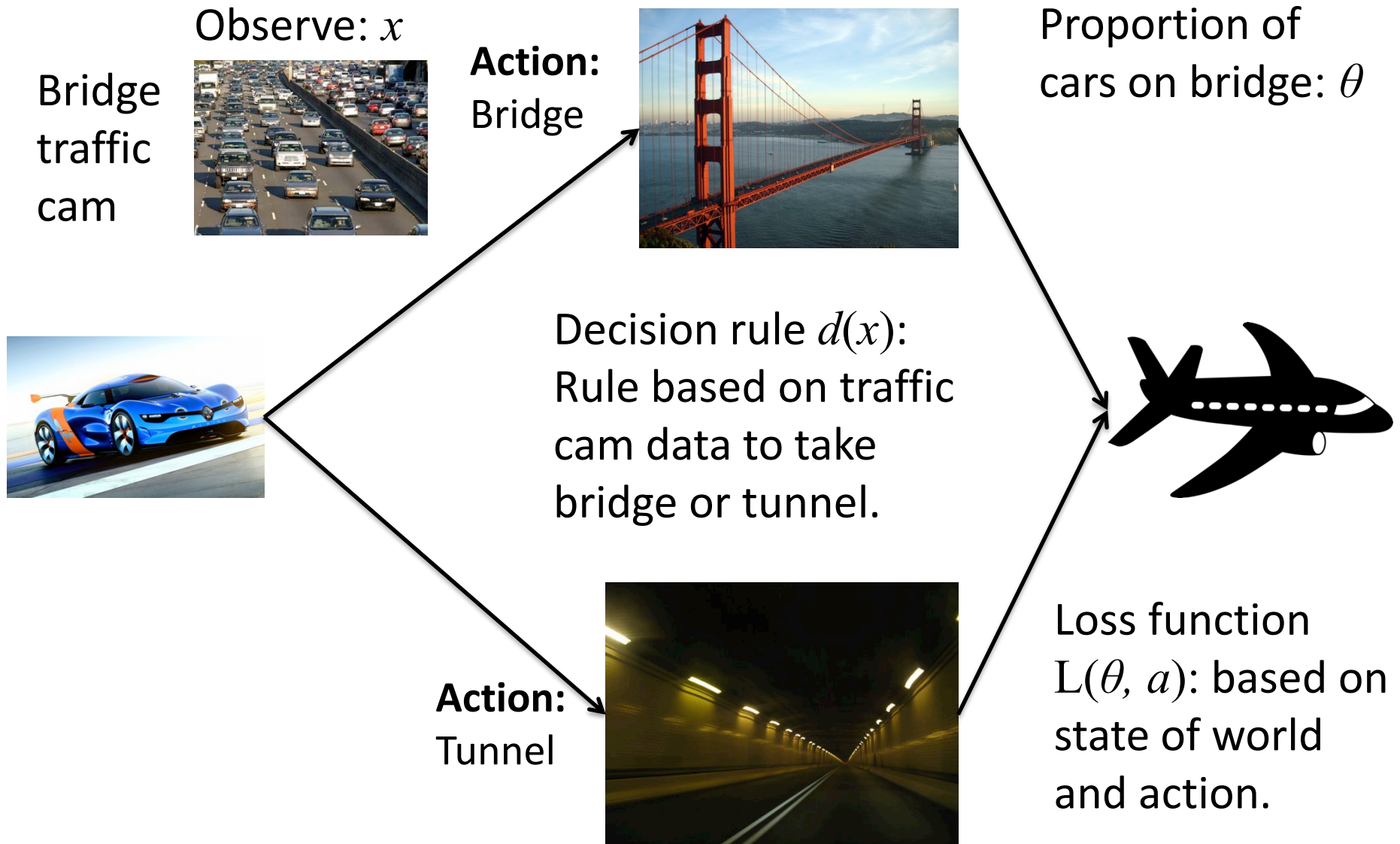
- Generalization Assignment grading a bit delayed.
 - Metareviews for cogsci + illness blindsided me.
- Thank you for turning in the project proposal.
 - Feedback went back to you this weekend
 - Prioritized this to make sure everyone got timely feedback
- Deadlines for assignments 3 and 4 pushed back a week:
 - Assignment 3 is due April 9 at 8:59pm
 - Assignment 4 is due April 23 at 8:59pm
- Questions?

- Admin Matters
- Statistical Decision Theory
- Markov Decision Processes
- **Short rest 1**
- Three student presentations:
 1. Schulz et al 1997 by Ashwini
 2. Ho et al 2015 by Khailanii

Short rest 2

3. Ho et al 2017 by Anirudh

Decisions under uncertainty



Statistical decision theory

Decision making under uncertainty formalism:

1. States of world θ (e.g., prop. of traffic on bridge)
2. Observation x (e.g., current traffic leading to bridge).
3. Possible actions A (e.g., take tunnel or bridge).
4. Decision rule $d(x)$ (e.g., take tunnel if x is heavy).
5. Loss function $L(\theta, a) = L(\theta, d(x))$
 1. 0-1 loss: 1 for wrong action, 0 for correct action. (e.g., 0 if you take bridge and less traffic on bridge)

For continuous “actions” (estimation):

1. Squared loss (also called L^2 norm): $(\theta - d(x))^2$
2. Absolute loss (also called L^1 norm): $|\theta - d(x)|$

Statistical decision theory

How “good” is your rule for deciding from the traffic cam based on the true percent of cars on the bridge?

Risk function: $R(\theta, d(x)) = E^x[L(\theta, d(x))]$

The risk function provides the expected loss for a decision rule $d(x)$ and a state of the world θ .

How should one pick a decision rule to minimize risk?

Statistical decision theory

Risk function: $R(\theta, d(x)) = E^x[L(\theta, d(x))]$

How should one pick a decision rule to minimize risk?

Do we have knowledge about the true state of the world?

If yes, can use a Bayesian criterion: minimize expected loss given a prior over the state of the world.

$$\operatorname{argmin}_{d(x)} E^\theta E^x[L(\theta, d(x))]$$

If no, can use a minimax criterion: minimize the worst possible loss over all possible states of the world.

$$\operatorname{argmin}_{d(x)} \max_{\theta} E^x[L(\theta, d(x))]$$

Statistical decision theory

Bayesian criterion: minimize expected loss given a prior over the state of the world.

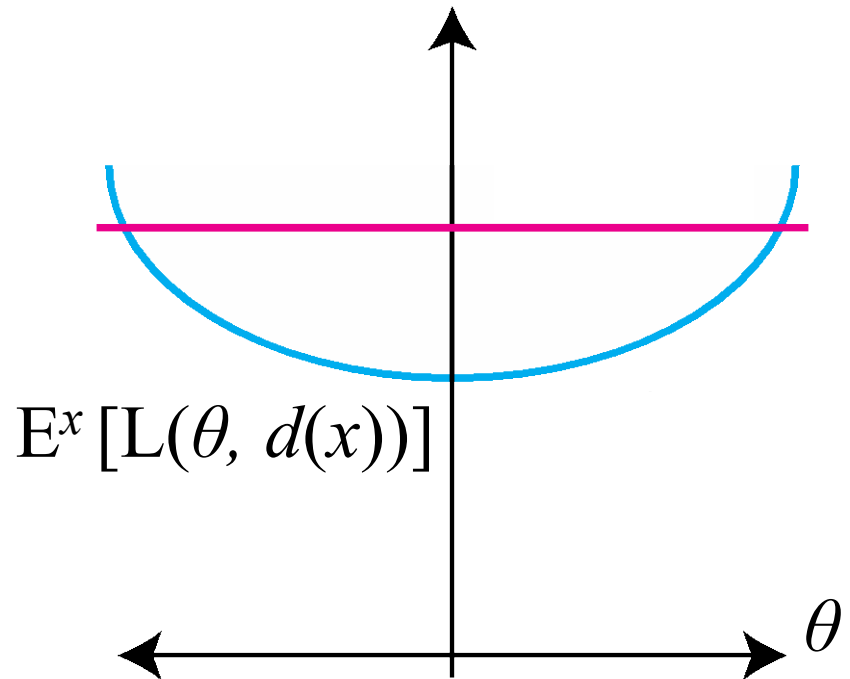
$$\operatorname{argmin}_{d(x)} E^{\theta} E^x[L(\theta, d(x))]$$

Minimax criterion: minimize the worst possible loss you could have over all possible states of the world.

$$\operatorname{argmin}_{d(x)} \max_{\theta} E^x[L(\theta, d(x))]$$

Exploring the difference
between the two criteria
graphically...

In the Bayesian case,
assume a uniform prior.



Statistical decision theory

Under Bayesian criterion with typical priors, setup, and given loss function,

0-1 loss: 1 for wrong action, 0 for correct action. (e.g., 0 if you take bridge and less traffic on bridge)

MAP is optimal

Squared loss (also called L^2 norm): $(\theta - d(x))^2$

Mean is optimal

Absolute loss (also called L^1 norm): $|\theta - d(x)|$

Median is optimal

- Admin Matters
- Statistical Decision Theory
- Markov Decision Processes
- **Short rest 1**
- Three student presentations:
 1. Schulz et al 1997 by Ashwini
 2. Ho et al 2015 by Khailanii

Short rest 2

3. Ho et al 2017 by Anirudh

Multiple actions

Statistical decision theory prescribes how we should make an action, but what about an action sequence?

In theory, no real difference.

“Action variable” takes action sequence as value.

But then how do we choose the correct sequence?

Work is in specifying the state dynamics, utilities/loss, and a rule for selecting the optimal sequence of actions.

Probably won't just be sum of rewards.

Want to **discount** future rewards.

Would you prefer \$10 now or \$10 in 1000 years?

Markov Chains + Decisions + Rewards = Markov Decision Processes (MDPs)

MDP defined by the following variables:

S = a set of “environmental” states

A = a set of possible actions for the agent to take
(decision rules in statistical decision theory).

T = *state transition function*, which defines a probability transition matrix for each action.

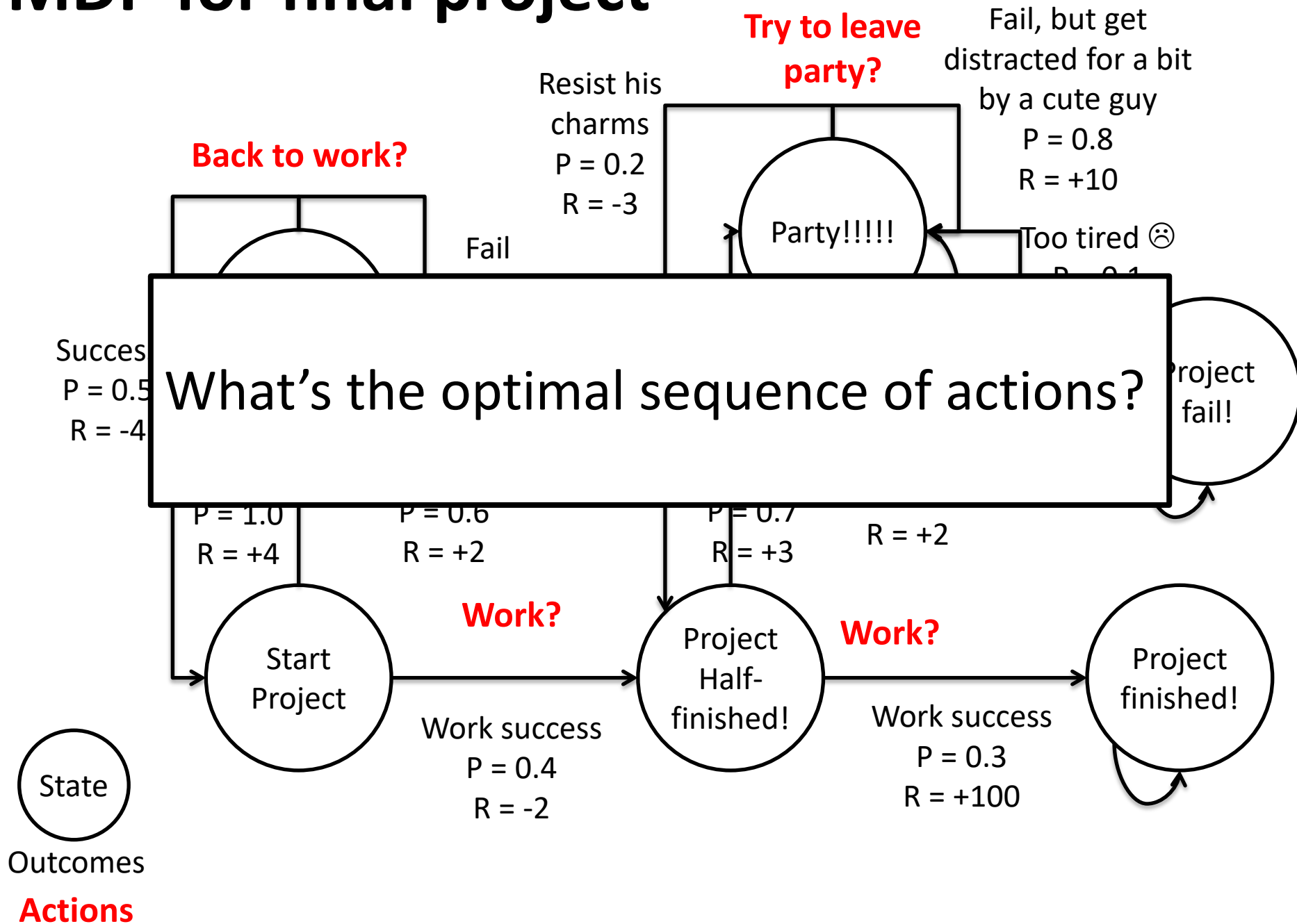
Typically this is written as $P(s_{t+1} | a_t, s_t)$

(MCs can be thought of as a special case with only 1 action)

R = a *reward function*. Maps states and actions, $R(s, a)$, to a reward (a real number). Reward = - Loss

γ = delayed discount parameter

MDP for final project



Policies, Values and Actions in MDPs

The actions an agent takes in each state is represented by a **policy** π , which is a distribution over actions in each state.

$$\pi = P(a_t | s_t)$$

Because it is Markov, it only depends on current state.

How do we count (and discount) future rewards?

Let $0 \leq \gamma \leq 1$ be the discount factor.

Let G_t be the total future discounted reward.

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The state-value function, $v_{\pi}(s) = E_{\pi}[G_t | S_t = s]$, is the expected reward from state s following policy π .

The action-value function, $q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$, is the expected return of taking action a from state s and then following policy π .

Policies, Values and Actions in MDPs

The optimal sequence of actions in a MDP can then be defined as the policy π^* satisfying

$$\pi^* = \arg \max_{\pi} v_{\pi}(s) = \arg \max_{\pi} E_{\pi}[G_t | S_t = s]$$

Reinforcement learning methods are methods for finding π^*

$$\begin{aligned} q_{\pi}(s, a) &= E_{\pi}[G_t | S_t = s, A_t = a] \\ &= E_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \end{aligned}$$

This sets up a recursive definition, which can be exploited using dynamic programming and iterative updates.

Different learning rules defined via different ways of resolving the random variables S_{t+1} and A_{t+1} .

iterative updates based on reinforcement during exploration is reinforcement learning (RL).

Max = Q-learning; Lots of elaborations from there...

Reinforcement Learning

$$\begin{aligned} q_{\pi}(s, a) &= E_{\pi}[G_t | S_t = s, A_t = a] \\ &= E_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \end{aligned}$$

Q-learning ($0 < \alpha < 1$ & $0 < \gamma < 1$)

Q_0 is a $S \times A$ matrix of q-values $q_{s,a} = q(s, a)$

Initialize Q_0 to 0^*

Select an action. How?

A temporal difference algorithm

Class of algorithms that learn the values of each state by updating whenever predicted value differs from observed value.

3. Update Q:


$$Q_{t+1} = Q_t + \alpha \left[r_{t+1} + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right]$$

Path	Path	Goal
Path	Garden	Garden
Start	Garden	Garden

```
possXs = 1:3;
```

```
possYs = 1:3;
```

```
actions = [up = 1; down = 2; right = 3; left = 4];
```

```
Q = zeros(length(possYs),length(possXs),length(actions));
```

```
Q = fillImpossVals(Q);
```

```
discRate = 0.95; % (gamma)
```

```
learnRate = 0.9; % (alpha)
```

```
randActProb = 0.1 % epsilon
```

```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [1, 1];

```

Path	Path	Goal
Path	Garden	Garden
Start X →	Garden	Garden

```

%1. What are the current actions
% and how good do we believe they are?
    [Up Right];    Q([cur_state, (Up, Right)]) = [0 0];
%2. What action do we take?
    Random Uniform[0,1] -> 0.16 < 0.9 % YES!
    %Which one? ceiling(2*Uniform) = ceiling(1.91) = 2 %RIGHT
%3. Take action, get feedback, and update Q-value/state
    new_state <- [1, 2];
    % Max Q-value of states next to new_state?    0
    % Feedback [-10] (in Garden)
    Q([cur_state, R]) += learnRate * (fb + discRate * maxQNext
                                     - Q([cur_state, R]));
    Q([cur_state, R) = 0 + .9*(-10 + .95*0-0) = -9

```

```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [1, 2];

```

Path	Path	Goal
Path	Garden	Garden
Start 	Garden X 	Garden

%1. What are the current actions

% and how good do we believe they are?

```
[U L R]; Q([cur_state, (U,L,R)]) = [0, 0, 0];
```

%2. What action do we take?

```
Random Uniform[0,1] -> 0.74 < 0.9 % Optimal (U,L,R)
```

```
%Which one? ceiling(3*Uniform) = ceiling(2.3) = 3 %RIGHT
```

%3. Take action, get feedback, and update Q-value/state

```
new_state <- [1, 3];
```

```
% Max Q-value of states next to new_state? 0
```

```
% Feedback [-10] (in Garden)
```

```
Q([cur_state, R]) += learnRate * (fb + discRate * maxQNext
                                - Q([cur_state, R]));
```

```
Q([cur_state, R]) = 0 + .9*(-10 + .95*0-0) = -9
```

```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [1, 3];

```

Path	Path	Goal
Path	Garden	Garden
Start 	Garden 	Garden X 

%1. What are the current actions

% and how good do we believe they are?

```
[U L]; Q([cur_state, (U, L)]) = [0, 0];
```

%2. What action do we take?

```
Random Uniform[0,1] -> 0.14 < 0.9 % Optimal (U,L)
```

```
%Which one? ceiling(2*Uniform) = ceiling(0.78) = 1 U
```

%3. Take action, get feedback, and update Q-value/state

```
new_state <- [2, 3];
```

```
% Max Q-value of states next to new_state? 0
```

```
% Feedback [-10] (in Garden)
```

```
Q([cur_state, U]) += learnRate * (fb + discRate * maxQNext
                                - Q([cur_state, U]));
```

```
Q([cur_state, U]) = 0 + .9*(-10 + .95*0-0) = -9
```

```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [2, 3];

```

Path	Path	Goal
Path	Garden	Garden ← X
Start →	Garden →	Garden ↑

```

%1. What are the current actions
% and how good do we believe they are?
[U, D, L]; Q([cur_state, (U, D, L)]) = [0, 0, 0];
%2. What action do we take?
Random Uniform[0,1] -> 0.42 < 0.9 % Optimal (U, D, L)
%Which one? ceiling(3*Uniform) = ceiling(2.11) = 3 %LEFT
%3. Take action, get feedback, and update Q-value/state
new_state <- [2, 2];
% Max Q-value of states next to new_state? 0
% Feedback [-10] (in Garden)
Q([cur_state, L]) += learnRate * (fb + discRate * maxQNext
                                - Q([cur_state, L]));
Q([cur_state, L]) = 0 + .9*(-10 + .95*0-0) = -9

```

```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [2, 2];

```

Path	Path	Goal
Path	Garden X 	Garden 
Start 	Garden 	Garden 

%1. What are the current actions

% and how good do we believe they are?

```
[U,D,L,R];    Q([cur_state, (U,D,L,R)]) = [0,0,0,0];
```

%2. What action do we take?

```
Random Uniform[0,1] -> 0.96 < 0.9 % SUBOPTIMAL (U,D,L,R)
```

```
%Which one? ceiling(4*Uniform) = ceiling(1.07) = 2 %D
```

%3. Take action, get feedback, and update Q-value/state

```
new_state <- [1, 2];
```

```
% Max Q-value of states next to new_state?    0
```

```
% Feedback [-10] (in Garden)
```

```
Q([cur_state, D) += learnRate * (fb + discRate * maxQNext
                                - Q([cur_state, D]));
```

```
Q([cur_state, D) = 0 + .9*(-10 + .95*0-0) = -9
```

```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [1, 2];

```

Path	Path	Goal
Path	Garden 	Garden 
Start 	Garden  X 	Garden 

```

%1. What are the current actions
% and how good do we believe they are?
    [U,L,R];    Q([cur_state, (U,L,R)]) = [0,0,-9];
%2. What action do we take?
    Random Uniform[0,1] -> 0.05 < 0.9 % OPTIMAL (U,L)
    %Which one? ceiling(3*Uniform) = ceiling(0.19) = 1 %U
%3. Take action, get feedback, and update Q-value/state
    new_state <- [2, 2];
    % Max Q-value of states next to new_state?    0
    % Feedback [-10] (in Garden)
    Q([cur_state, U]) += learnRate * (fb + discRate * maxQNext
                                     - Q([cur_state, U]));
    Q([cur_state, U) = 0 + .9*(-10 + .95*0-0) = -9

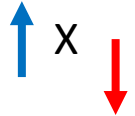
```



```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [2, 2];

```

Path	Path	Goal
Path	Garden 	Garden 
Start 	Garden 	Garden 

%1. What are the current actions

% and how good do we believe they are?

```

[U,D,L,R];    Q([cur_state, (U,D,L,R)]) = [0,-9,0,0];

```

%2. What action do we take?

```

Random Uniform[0,1] -> 0.82 < 0.9 % OPTIMAL (U,L,R)

```

```

%Which one? ceiling(3*Uniform) = ceiling(0.79) = 1 %U

```

%3. Take action, get feedback, and update Q-value/state

```

new_state <- [3, 2];

```

```

% Max Q-value of states next to new_state?    0

```

```

% Feedback  [+10] (onto Path)

```

```

Q([cur_state, U]) += learnRate * (fb + discRate * maxQNext
                                - Q([cur_state, U]));

```

```

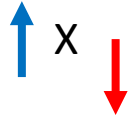
Q([cur_state, U) = 0 + .9*(-10 + .95*0-0) = 9

```

```

discRate = 0.95; % (gamma)
learnRate = 0.9; % (alpha)
randActProb = 0.1; % epsilon
cur_state = [3, 2];

```

Path	Path X	Goal
Path	Garden X 	Garden 
Start 	Garden 	Garden 

And so on...

Reinforcement Learning

$$Q_{t+1} = Q_t + \alpha \left[r_{t+1} + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right]$$

Q-learning updates taking action that gives largest expected value according to current estimates.

Lots of variants -> lots of similar, but slightly different RL methods (e.g., SARSA).

“Model-free” because we do not propagate updates throughout the state space.

“Model-based” methods propagates reward signals beyond the immediate state based on what states can get to the immediate state in one (two, three, etc.) step(s).

Reinforcement Learning

“Model-free” because we do not propagate updates throughout the state space.

“Model-based” methods propagates reward signals beyond the immediate state based on what states can get to the immediate state in one (two, three, etc.) step(s).

Why does this matter?

In real-world applications, we don't know the transition and reward functions.

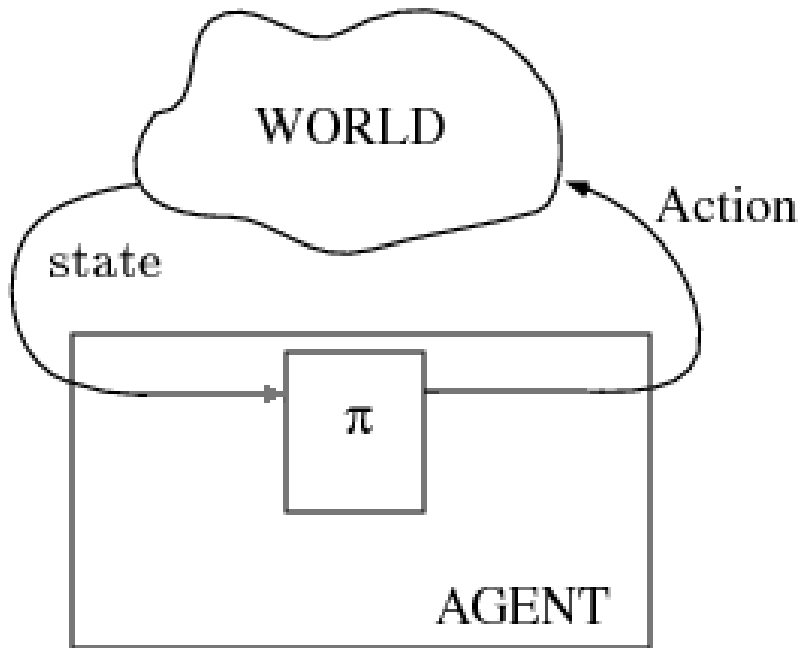
If we did, we could just sit and simulate to compute the optimal policy before acting! => no learning

We estimate those as we go. If we do, we get model-based approaches...

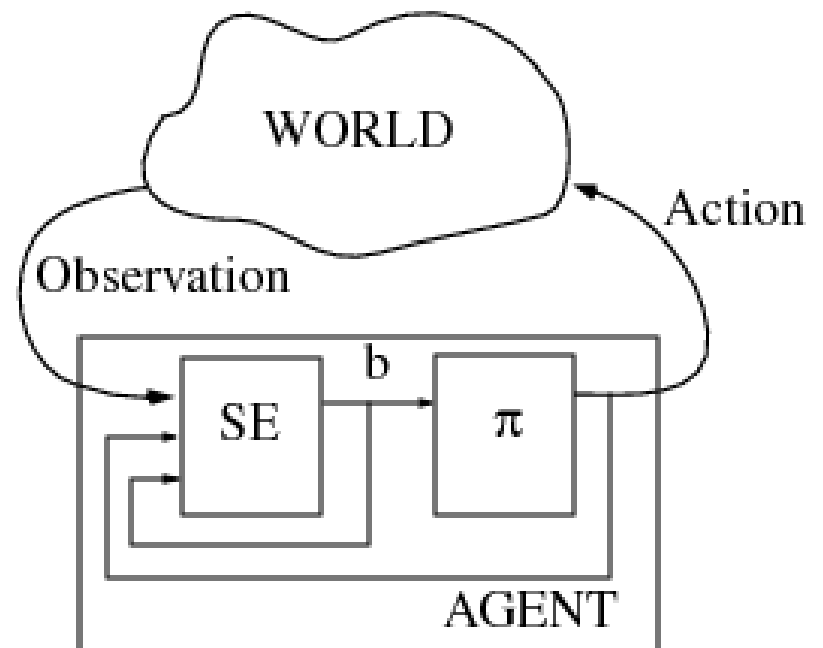
What if there are parts of the state
(or rewards or transitions) that we
do not observe?

Enter: Partially Observable Markov
Decision Processes (POMDPs)

MDP setup



POMDP setup



Can we convert one into the other?

Yes! A POMDP can be turned into an MDP, where belief over states (in the POMDP) are the states of the equivalent MDP.

This is one way to solve POMDPs, but in general, it is really hard...

<http://www.pomdp.org/> is a useful resource for code and tutorials...

- Admin Matters
- Statistical Decision Theory
- Markov Decision Processes
- **Short rest 1**
- Three student presentations:
 1. Schulz et al 1997 by Ashwini
 2. Ho et al 2015 by Khailanii

Short rest 2

3. Ho et al 2017 by Anirudh

Short Rest 1



- Admin Matters
- Statistical Decision Theory
- Markov Decision Processes
- **Short rest 1**
- Three student presentations:
 1. Schulz et al 1997 by Ashwini
 2. Ho et al 2015 by Khailanii

Short rest 2

3. Ho et al 2017 by Anirudh